

喜马拉雅小程序插件 v2.0.4

喜马拉雅儿童内容 SDK 插件

版本：v2.3.3 | 更新日期：2026-08-31

喜马拉雅儿童内容 SDK 插件（小喜马）可让合作方小程序快速接入喜马拉雅儿童音频内容，提供完整的内容浏览、搜索、播放与会员体验。

一、接入前准备

事项	说明
获取插件 AppID	联系喜马拉雅侧分配
获取合作方 AppKey	喜马拉雅分配，用于标识合作方身份
微信开放平台权限	确认宿主小程序已开启"使用插件"能力
设备 ID (deviceId)	由合作方生成/维护，标识终端设备（可选）
RSA 加密公钥（静默登陆）	喜马拉雅提供，用于加密用户手机号生成静默登陆 ticket（可选，仅需静默登陆时申请）

申请插件权限路径：

微信公众平台 → 登录小程序 → 账号设置 → 第三方设置 → 插件管理 → 搜索添加插件

完成添加后通知喜马拉雅人员通过申请（申请仅 24 小时有效）。

申请半屏小程序权限路径：

微信公众平台 → 登录小程序 → 账号设置 → 第三方设置 → 半屏小程序管理 → 申请喜马拉雅小程序权限

二、插件注册

在宿主小程序的 `app.json` 中添加以下声明：

```
{
  "plugins": {
    "xmllyPlugin": {
      "version": "1.0.0",
      "provider": "<插件AppID>",
      "export": "plugin-exports.js"
    }
  },
  "embeddedAppIdList": ["<喜马拉雅小程序AppID>"]
}
```

`"export"` 字段声明宿主向插件暴露的接口文件，插件通过 `requireMiniProgram()` 调用其中的方法获取 VIP 票据等数据，详见第六节。

体积大时不要放主包顶层。上面这种把插件声明在 `app.json` 顶层 `plugins` 的做法，会让插件代码随主包一起下载，直接增加主包体积（微信小程序主包有 2M 上限）。如果插件较大，应改用下面的【2.1 分包加载插件】把它放进分包，再配【2.2 分包预加载】提升进入速度。

2.1 分包加载插件（插件体积大时）

微信官方支持在分包内引入插件。将插件的 `plugins` 声明放到 `subPackages` 的某个分包里，插件代码只随该分包下载，不占主包体积。

```
{
  "pages": [
    "pages/index/index"
  ],
  "subPackages": [
    {
      "root": "packages/plugin-host",
      "name": "pluginHost",
      "pages": [
        "pages/demo/demo"
      ],
      "plugins": {
        "xmllyPlugin": {
          "version": "1.0.0",
          "provider": "<插件AppID>",
          "path": "../dist/plugin/plugin",
          "export": "plugin-exports.js"
        }
      }
    }
  ]
}
```

```

    }
  }
},
"embeddedAppIdList": ["<喜马拉雅小程序AppID>"]
}

```

分包内插件的限制（务必阅读）：

限制	说明
仅限本分包使用	默认只能在该定义插件的分包内使用，跨分包需配合【分包异步化】
插件不可被多个分包共享	同一个插件不能被多个分包同时引用
低基础库跳转限制	基础库 < 2.9.0 时，不能从分包外直接跳入分包内插件页面，需先跳入分包内非插件页面再转入插件页
<code>export</code> 相对分包根	分包中声明插件时， <code>export</code> 文件路径相对该分包根，而非项目根

分包与主包、预下载的关系：

- 普通分包依赖主包，不能反向预下载主包；只有**独立分包**（`"independent": true`）才能预下载主包。
- 上面的 `pluginHost` 是普通分包，作为**被预下载方**使用。

2.2 分包预加载（加快分包加载速度）

`preloadRule` 让框架在进入某个页面时提前下载需要的分包，减少用户进入后续分包页面时的白屏等待。只支持配置，不能通过 API 调用。

```

{
  "pages": [
    "pages/index/index"
  ],
  "preloadRule": {
    "pages/index/index": {
      "packages": ["pluginHost"],
      "network": "all"
    }
  }
}

```

复制代码

```
}  
}
```

字段说明：

字段	类型	必填	默认值	说明
<code>key</code> （页面路径）	string	是	无	进入此页面时触发预下载
<code>packages</code>	StringArray	是	无	要预下载分包的 <code>root</code> 或 <code>name</code> ； <code>__APP__</code> 表示主包
<code>network</code>	string	否	<code>wifi</code>	预下载网络： <code>all</code> 不限 / <code>wifi</code> 仅 wifi

注意事项：

- 同一个分包内的页面共享预下载总量 **2M**，超过会在打包时校验报错。
- 上例在主包页 `pages/index/index` 进入时预下载 `pluginHost` 分包，`network: all` 表示 wifi / 流量都预下载；若想省流量可省略或在 `wifi` 下才预下载。
- 验证方式：微信开发者工具打开后进入 `pages/index/index`，vConsole 出现 `preloadSubpackages` 开头的日志即表示预下载触发。

三、可用页面

宿主小程序可跳转的页面，所有路径均须携带 `appKey` 参数：

页面	路径	必填参数	可选参数
首页	<code>plugin://xmlyPlugin/home</code>	<code>appKey</code>	<code>deviceId</code>
搜索结果	<code>plugin://xmlyPlugin/search-result</code>	<code>appKey</code>	<code>val</code> （关键词，需 <code>encodeURIComponent</code> ）

专辑详情	plugin://xmlyPlugin/album-detail	appKey 、 albumId	trackId 、 deviceId
个人中心	plugin://xmlyPlugin/personal-center	appKey	deviceId
权益领取	plugin://xmlyPlugin/vip-benefit	appKey	deviceId

跳转示例

 复制代码

```
const plugin = requirePlugin('xmlyPlugin');

wx.navigateTo({
  url: 'plugin://xmlyPlugin/home?appKey=YOUR_APP_KEY&deviceId=YOUR_DEVICE_ID',
  events: {
    [plugin.ChannelEvents.PLUGIN_TO_HOST]: function(message) {
      handlePluginMessage(message);
    }
  },
  success: function(res) {
    this.eventChannel = res.eventChannel;
  }
});
```

四、连接参数

参数名	类型	必填	说明
appKey	string	是	喜马拉雅分配的合作方标识，缺少将导致数据归属异常


```
// message.data 结构
{
  albumId: string,      // 专辑 ID
  trackId: string,      // 单集 ID
  breakSecond: number,  // 断点续播秒数（首次为 0）
  duration?: number,    // 音频时长（秒）
  albumTitle?: string,  // 专辑名称
  trackTitle?: string,  // 单集名称
}
```

5.2 插件 → 宿主：登录完成通知

复制代码

```
// message.data 类型同 TokenInfo（定义见第七节）
{
  access_token: string | null,
  refresh_token?: string,
  uid?: string | number,
  expires_in?: number,
  scope?: string,
}
```

5.3 插件 → 宿主：用户信息更新通知

用户完成 VIP 购买或会员状态刷新后，插件自动推送此事件，宿主可据此同步最新 VIP 状态（如更新角标、刷新会员权益页）。

复制代码

```
// message.data 结构
{
  userInfo: {
    id: number,      // 用户 ID
    nickname: string, // 昵称
    avatar_url: string, // 头像 URL
    is_verified: boolean, // 是否认证
    is_vip: boolean, // 是否有喜马拉雅 VIP
    vip_expired_at: number, // 喜马拉雅 VIP 到期时间戳（秒）
    is_kid_vip: boolean, // 是否有儿童 VIP
    kid_vip_expire_date: number | null, // 儿童 VIP 到期时间戳（毫秒），未
    开通时为 null
  } | null // null 表示退出登录
}
```

```
}
```

5.4 消息处理示例

 复制代码

```
function handlePluginMessage(message) {
  switch (message.type) {
    case plugin.PluginMessageTypes.PLAY:
      myPlayer.play({
        albumId: message.data.albumId,
        trackId: message.data.trackId,
        startSecond: message.data.breakSecond,
        duration: message.data.duration,
        albumTitle: message.data.albumTitle,
        trackTitle: message.data.trackTitle,
      });
      break;
    case plugin.PluginMessageTypes.LOGIN_COMPLETE:
      storage.set('xmly_token', message.data.access_token);
      break;
    case plugin.PluginMessageTypes.USER_INFO_UPDATED:
      // message.data.userInfo 为 null 时表示退出登录
      if (message.data.userInfo) {
        updateUserInfo(message.data.userInfo)
      } else {
        clearVipStatus();
      }
      break;
    case plugin.PluginMessageTypes.LOGOUT:
      storage.remove('xmly_token');
      clearUserInfo();
      break;
  }
}
```

5.5 宿主 → 插件：通知拉起支付

 复制代码

```
this.eventChannel.emit(plugin.ChannelEvents.HOST_TO_PLUGIN, {
  type: plugin.HostMessageTypes.NEED_PAYMENT,
  version: '1.0',
  timestamp: Date.now(),
});
```

```
data: { title: '开通会员享完整内容' }, // 可选
});
```

六、宿主导出接口（plugin-exports.js）

插件通过 `requireMiniProgram()` 调用宿主提供的接口。宿主须在项目根目录创建 `plugin-exports.js`：

```
// plugin-exports.js
module.exports = {
  // 返回 VIP 权益票据（用于权益领取）
  getVipTicket() {
    return { ticket: 'YOUR_VIP_TICKET' };
  },

  // 返回第三方设备唯一标识（如有）
  getVipDeviceInfo() {
    return { device_id: 'YOUR_DEVICE_ID' };
  },

  // 返回静默登陆 ticket（可选，仅需静默登陆时实现）
  getAuthTicket() {
    return { ticket: 'YOUR_AUTH_TICKET' };
  },

  // 返回设备连接状态：0 = 在线，1 = 离线
  getDeviceConnectionStatus() {
    return yourDeviceManager.isConnected() ? 0 : 1;
  },
};
```

方法	返回值	说明
<code>getVipTicket()</code>	<code>{ ticket: string }</code>	VIP 权益票据，用于权益领取流程
<code>getVipDeviceInfo()</code>	<code>{ device_id: string }</code>	第三方设备 ID，可选；不实现时插件降级处理

<code>getAuthTicket()</code>	<code>{ ticket: string }</code>	静默登陆 ticket, 可选; 不实现时插件走普通登陆流程, 详见 6.1 节
<code>getDeviceConnectionStatus()</code>	<code>0 1</code>	当前设备连接状态: <code>0</code> = 在线, <code>1</code> = 离线; 用于播放前判断, 不实现时默认视为在线

6.1 静默登陆 (getAuthTicket)

前置准备:

资源	来源
用户手机号	合作方自有
RSA 加密公钥	喜马拉雅提供
用户已同意自动登陆协议	合作方在自有页面完成授权确认, 用户未同意时禁止调用静默登陆流程

ticket 生成流程:

 复制代码

用户手机号

- RSA 公钥加密 (PKCS#1 v1.5 或 OAEP, 以喜马拉雅提供的公钥为准)
- 密文转 Hex 编码
- 即为 ticket

手机号加密示例 (Node.js) :

 复制代码

```
const crypto = require('crypto');
```

/**

- * 生成静默登陆 ticket
- * @param {string} phone 用户手机号 (明文)
- * @param {string} publicKey 喜马拉雅提供的 RSA 公钥 (PEM 格式)
- * @returns {string} Hex 编码的密文, 直接作为 ticket 使用
- */

```
function generateAuthTicket(phone, publicKey) {
  const encrypted = crypto.publicEncrypt(
    {
      key: publicKey,
      padding: crypto.constants.RSA_PKCS1_PADDING,
    },
    Buffer.from(phone, 'utf8')
  );
  return encrypted.toString('hex');
}

// plugin-exports.js 中调用 (ticket 应由服务端下发, 不要在小程序端持有公钥)
module.exports = {
  async getAuthTicket() {
    const ticket = await fetchTicketFromYourServer(); // 向自有服务端请求
    return { ticket };
  },
};
```

七、插件导出接口 (requirePlugin)

∨
复制代码

```
interface TokenInfo {
  access_token: string | null;
  refresh_token?: string;
  uid?: string | number;
  expires_in?: number;
  scope?: string;
}

interface UserInfo {
  id: number;
  nickname: string;
  avatar_url: string;
  is_verified: boolean;
  is_vip: boolean;
  vip_expired_at: number;
}

interface PlatformInfo {
  platform: 'plugin';
  version: string;
}
```

```
}
```

[复制代码](#)

```
const plugin = requirePlugin('xmlyPlugin');

const tokenInfo = plugin.getTokenInfo(); // TokenInfo
const userInfo = plugin.getUserInfo(); // UserInfo | null, 未登录时为 null
const platformInfo = plugin.getPlatformInfo(); // PlatformInfo
```

方法	返回类型	说明
<code>getTokenInfo()</code>	<code>TokenInfo</code>	当前登录凭证； <code>access_token</code> 为 null 表示未登录
<code>getUserInfo()</code>	<code>UserInfo null</code>	当前登录用户信息；未登录时返回 null
<code>getPlatformInfo()</code>	<code>PlatformInfo</code>	插件平台标识及版本号

八、支付说明

插件内购买会员由插件侧处理，无需宿主实现支付逻辑：

平台	行为
Android	自动跳转喜马拉雅支付小程序
iOS（Apple 政策限制）	展示支付二维码，用户扫码完成支付

若宿主设备播放到付费内容需引导购买，通过第五节 5.4 的 `need_payment` 消息通知插件即可。

九、接入检查清单

☐ 已申请插件权限及半屏小程序权限

- ☐ `app.json` 已注册插件，包含 `"export": "plugin-exports.js"`
- ☐ （插件体积大时）插件已放入分包 `subPackages`，未占主包体积，且未违反「插件不可被多分包共享」限制
- ☐ （如需提升进入速度）已在 `app.json` 配置 `preloadRule` 预加载插件分包
- ☐ 已实现 `plugin-exports.js`，导出 `getVipTicket`、`getVipDeviceInfo`、`getDeviceConnectionStatus`
- ☐ （静默登陆）已向喜马拉雅申请 RSA 公钥，并在服务端实现 ticket 生成逻辑，`plugin-exports.js` 中导出 `getAuthTicket`
- ☐ 跳转插件页面时携带了 `appKey` 参数
- ☐ 使用 `plugin.ChannelEvents.PLUGIN_TO_HOST` 监听插件消息（未硬编码字符串）
- ☐ 已保存 `success` 回调中的 `eventChannel` 引用
- ☐ 处理 `LOGOUT` 消息，清理本地持久化的喜马拉雅登录态
- ☐ （如需读取用户信息）通过 `plugin.getUserInfo()` 获取，未登录时做 `null` 判断
- ☐ 已验证 iOS / Android 双端效果

十、参考资料

- [微信小程序插件使用文档](#)
- [微信半屏小程序文档](#)

Changelog

v2.3.3 — 2026-08-31

新增：插件分包加载与预加载

- 插件体积较大时，改为在分包 `subPackages` 中声明插件，降低主包体积（见 2.1 节）
- 新增 `preloadRule` 配置，进入主包页时预加载插件分包，加快进入插件速度（见 2.2 节）
- 接入检查清单补充分包加载与预加载检查项

v2.3.2 — 2026-08-31

新增：entryNonce 连接参数

- 新增 `entryNonce` 参数，用于解决插件生命周期不会随关闭而销毁导致的状态复用问题（见第四节）
 - 插件据此识别宿主"新一次进入"，重新执行登录检查与静默登录
-

v2.1.0 — 2026-07-17

新增：插件 → 宿主事件

- `LOGOUT`：用户在插件内主动退出登录时触发，宿主应清理本地持久化的喜马拉雅 token 及用户信息（见第五节 5.4）

新增：宿主导出接口

- `getDeviceConnectionStatus()`：返回当前设备连接状态（`0` = 在线，`1` = 离线）。插件在用户触发播放前调用；离线时 toast 提示"设备不在线"并阻止播放事件推送；未实现时默认视为在线（见第六节）
-

v2.0.0 — 初始版本

- 首页、搜索结果、专辑详情、个人中心、权益领取页面接入
- EventChannel 双向通信：`PLAY`、`LOGIN_COMPLETE`、`USER_INFO_UPDATED`、`NEED_PAYMENT`
- 宿主导出接口：`getVipTicket`、`getVipDeviceInfo`、`getAuthTicket`
- 插件导出接口：`getTokenInfo`、`getUserInfo`、`getPlatformInfo`